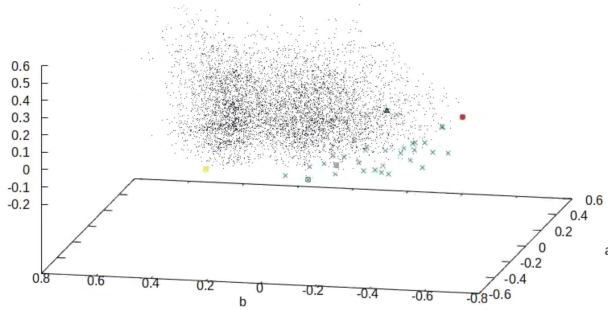
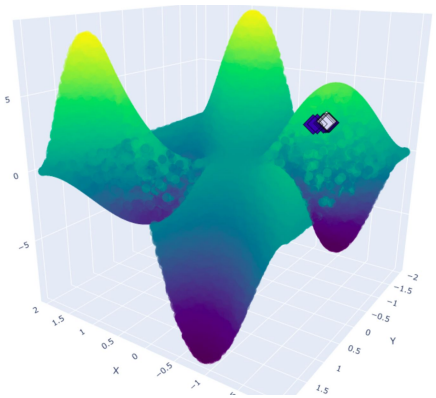


SMLP



Constrained Multi-Objective Optimization Engine:

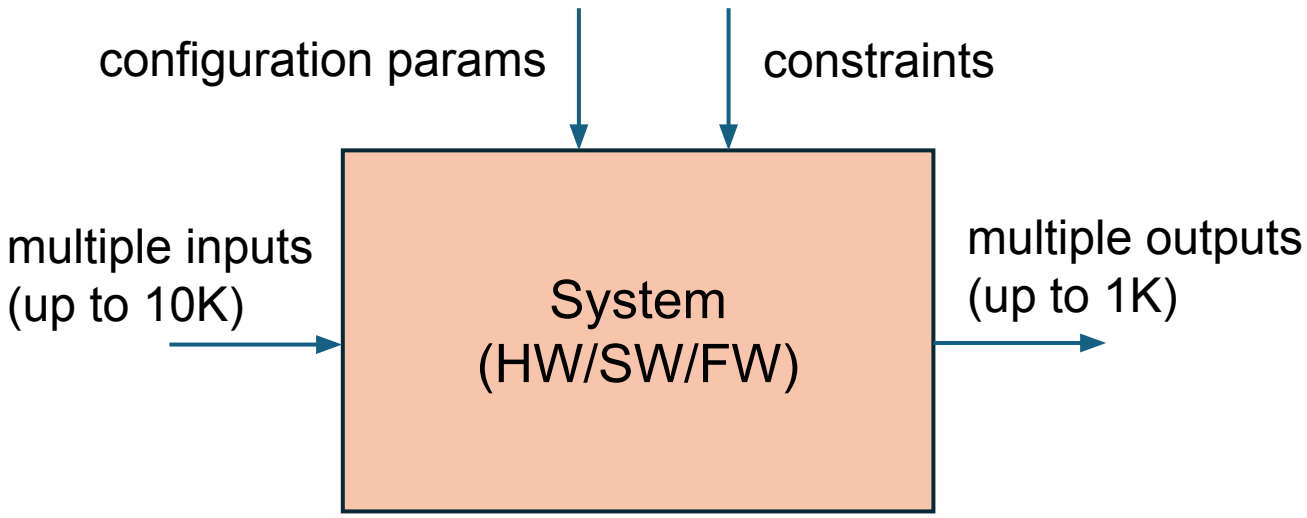
system optimization, expiration, verification, synthesis, root-causing

AI meets Formal

F. Brauße, Z. Khasidashvili, K. Korovin, D. Messerman

SMLP

Find optimal configuration

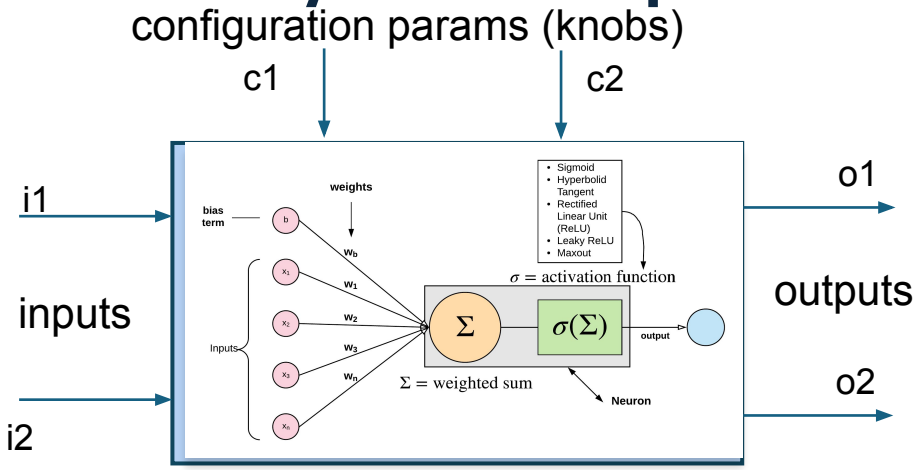


Get quick and accurate insights about your system:

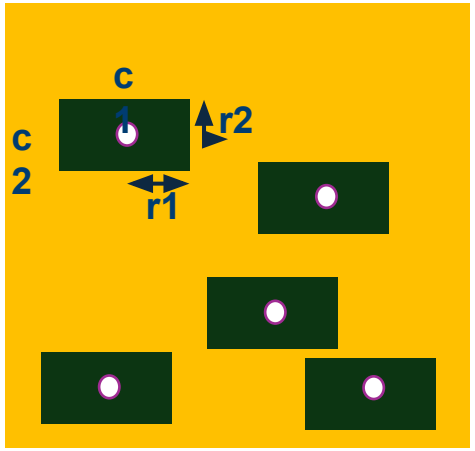
1. Optimization: What are the optimal system configurations?
2. Verification: Does my system satisfy the requirements?
3. Root-causing: What are input ranges that violate the spec?
4. Exploration: Which parameters are the most important?

SMLP

Safety and Optimality with Stability

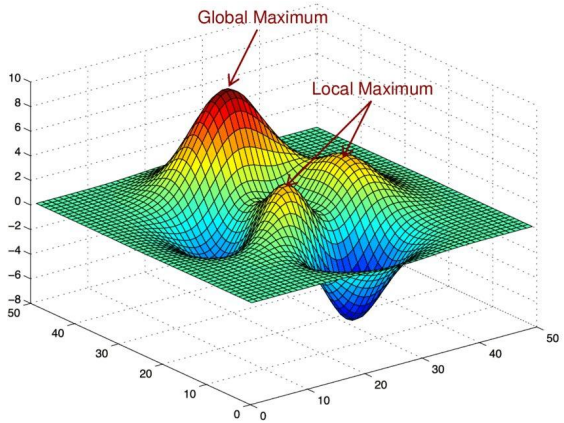


Robustness (stability)

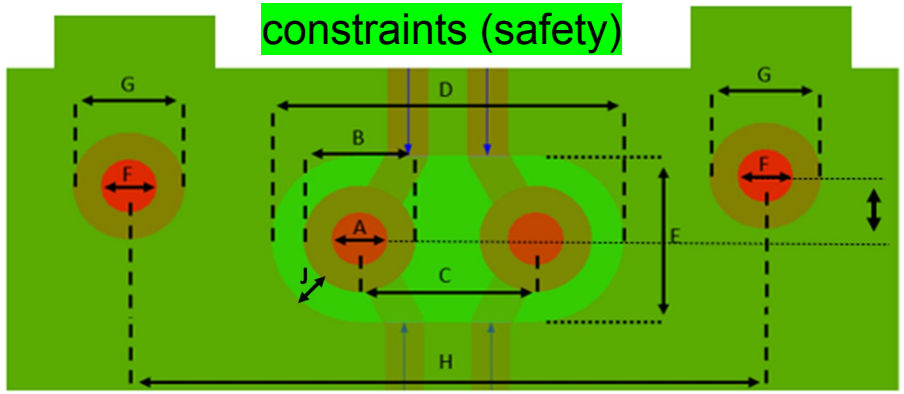


- Some optimization applications require the selected configurations to (1) satisfy constraints (2) be robust (stable) against perturbations, and (3) be optimal:

(pareto) optimality

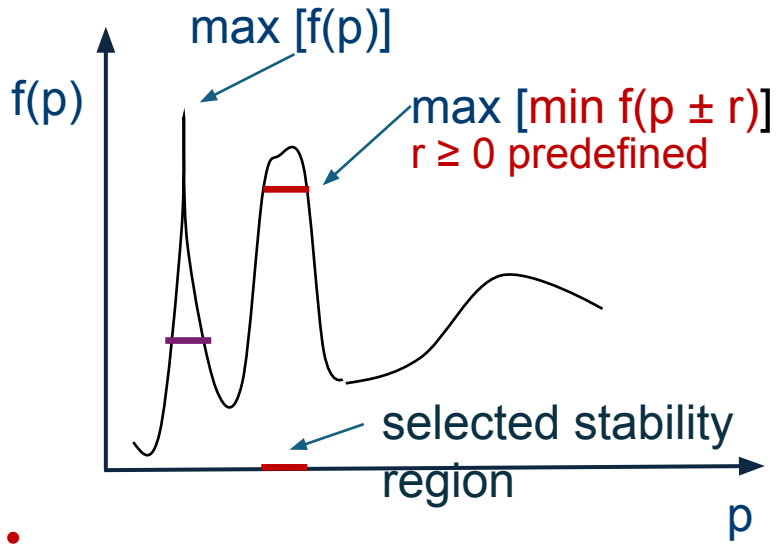


constraints (safety)



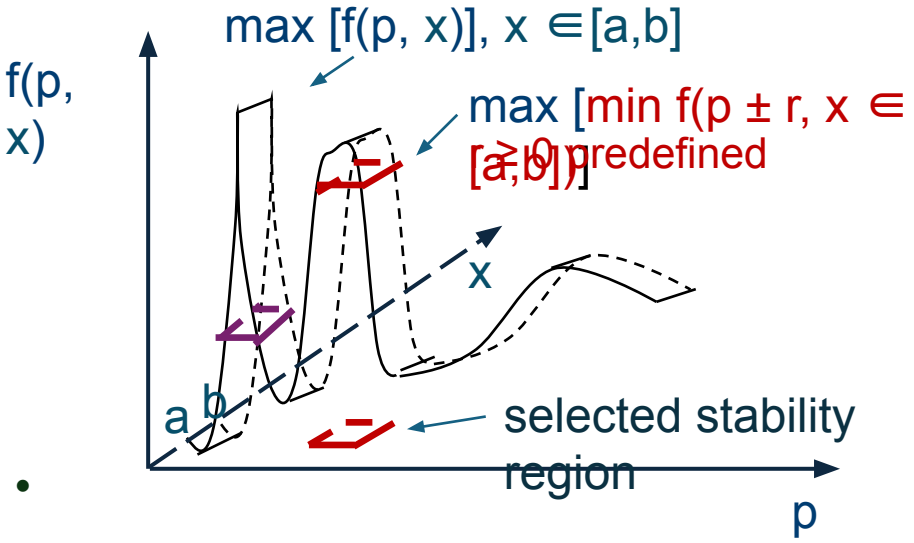
Safety and Optimality with Stability (avoid narrow peaks)

- Maximum f(c) of function f(p), with stability radius r: min-max optimization



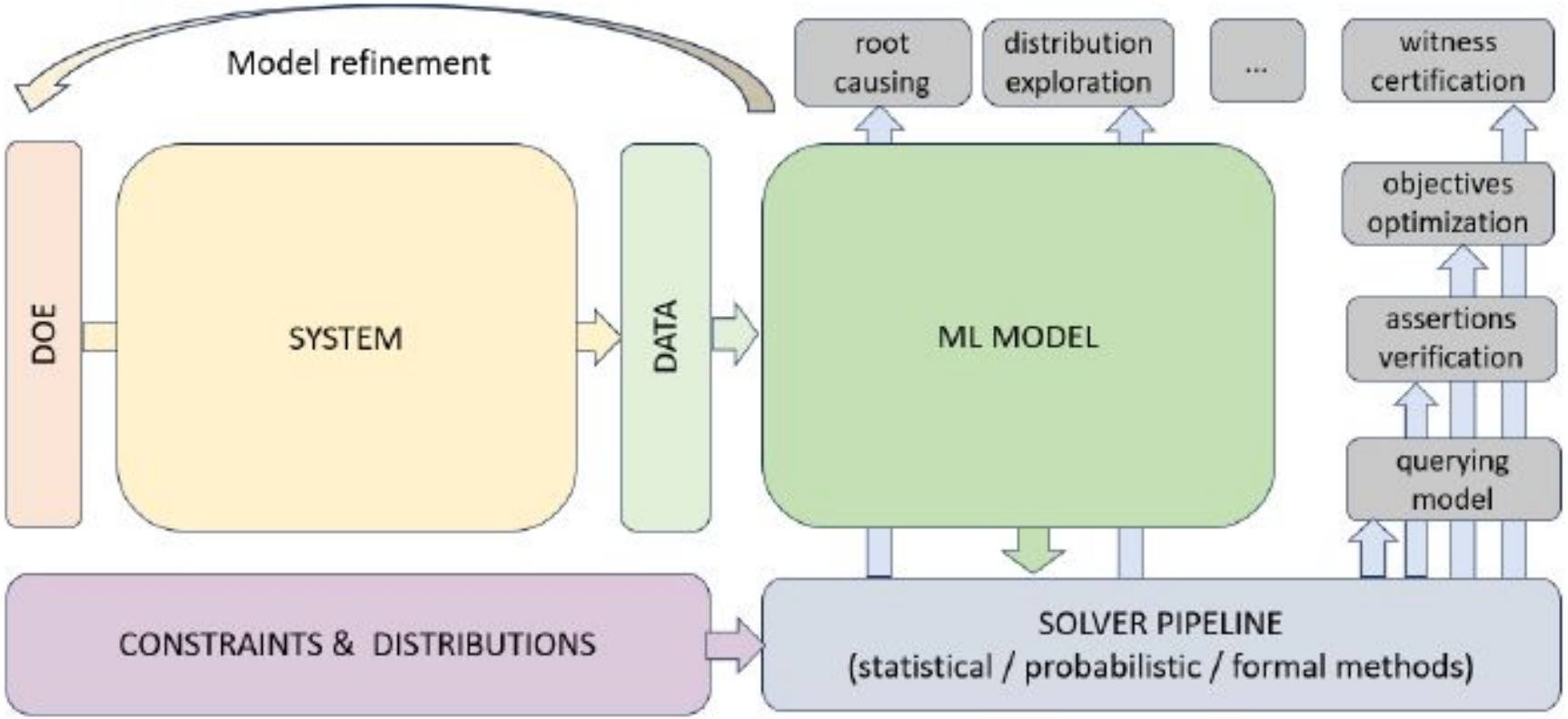
- The radius can be an absolute value of that parameter or relative to the value of the center point c (e.g. 5% of c)

- In addition, we consider parametrized f(p,x) and optimize parameters to find stable maximum f(c,x), for all inputs x.



- The free inputs x can be numeric (range of supported values) or categorical (supported modes of op)

How: SMLP functionality and architecture



SMLP GIT
https://github.com/SMLP-Systems/smlp

SMLP

Why SMLP

- Optimality guarantee:

  Computed configurations are guaranteed to be optimal.
- Specification is guaranteed:

  Specification is satisfied even under perturbations.
- Multi-objective optimization:

  Optimizing tradeoff between conflicting objectives.
- Robustness / stability:

  Optimization results are valid under perturbations: to account for noise in the data imprecise measurements, varying operating conditions, manufacturing variations, or adversarial attacks.

SMLP

Use case examples

Successfully applied to:

- Signal integrity: IO configuration settings optimization for DDR.
- Circuit board design: via layout optimization for multiple stackups.
- Hardware optimization for performance, area and power tradeoff.
- BIOS MRC configuration selection in EV

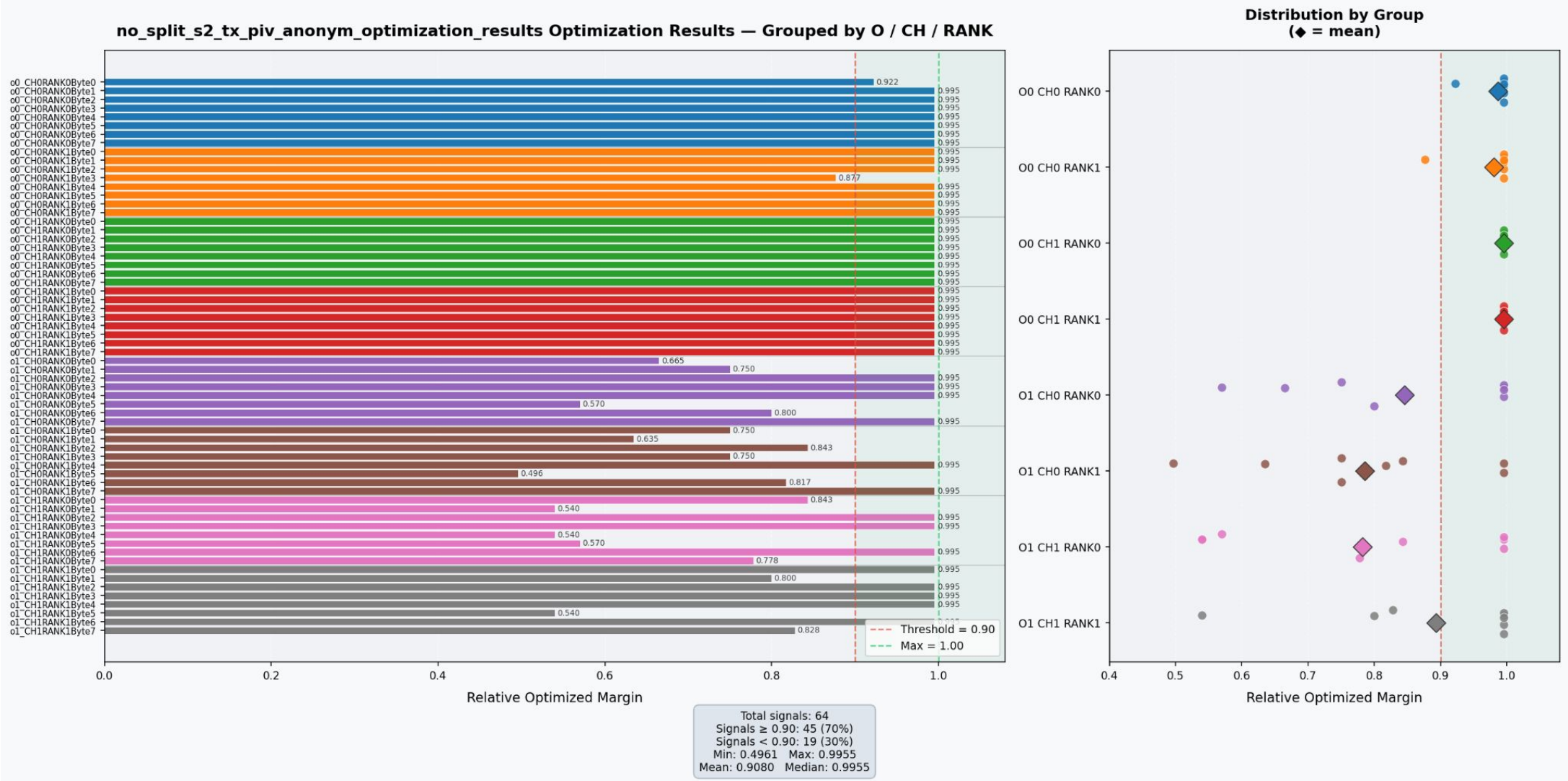
In future:

- Analog module configuration optimization, validation, root-cause analysis
- Board layout optimization with Design for Manufacturing constraints
- Applications in 2.3D packaging – configuration and validation
- Kubernetes cluster load balancing optimization.
- HPC (High Performance Computing) Networks load optimizations.
- Multi-Process Service (MPS) optimization.
- Analysis and optimization of ML models, LLMs, NLP, security.

SMLP

Intel Signal Integrity MultiObjective Optimization

- CPU-DDR bus Pareto Optimization for 64 bits (*)



(*) Names and numbers are obfuscated

Easy to integrate

- Command line: available at product quality.
- Via Python API: available at product quality.
- Net-batch API: Multiple pareto solutions are presented to user.
- Via GUI: developed, towards production.
- As an AI Agent : SMLP Agent is under development.
- As endpoint tool: MCP interface is under development.

SMLP

Deep dive in SMLP technology

- SMLP GIT https://github.com/SMLP-Systems/smlp
- SMLP Manual: SMLP: Symbolic Machine Learning Prover (User Manual)
- CAV 2024: [2402.01415] SMLP: Symbolic Machine Learning Prover
- IJCAI 2022: Combining Constraint Solving and Bayesian Techniques for System Optimization.
- FMCAD 2020: Selecting stable safe configurations for systems modelled by neural networks with ReLU activation

SMLP

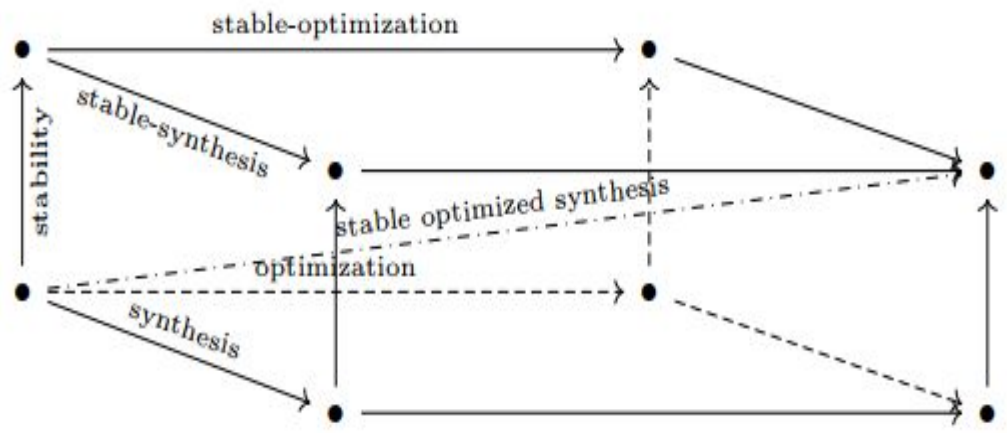
Extra details

SMLP analysis capabilities

- SMLP is a tool form ML model exploration:
  - Design of Experiments (DOE) for data generation form systems
  - Data processing
  - Training models, prediction
  - Model verification – checking the model against assertions
  - Model configuration optimization with respect to optimization objectives
  - Querying model – finding input vectors satisfying queries (task opposite to verification)
  - Model optimized synthesis – optimizing configuration so that assertions are satisfied
  - Model refinement loop to close the gap with the system it models
  - UPM/DPM analysis (more accurate / robust compared to current methodology)
  - Feature selection and range analysis
  - Distribution analysis
  - More…

SMLP modes

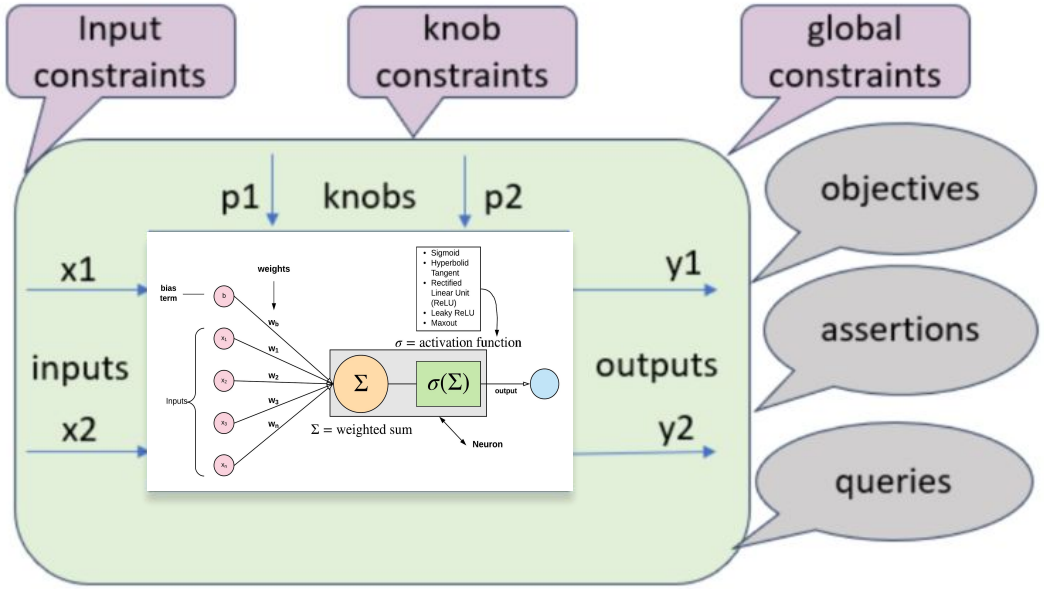
- Model Exploration Modes
  - 'certify': stable configuration certification
  - 'optimize: stability optimization with constraints
  - 'synthesize': stable synthesis with constraints
  - 'query': generate stable witness
  - 'verify': assertion verification with stability (check if stable witness exists ti negated assertion)
  - 'optsyn': combines optimization and synthesis (finds optimal configurations where assertions are also valid)
- DOE (Design of Experiments)
- Regression model training and prediction (NN, tees, polynomial models)
- Feature Selection/Ranges Analysis (in progress)



Model Exploration cube

SMLP Problem specification (optimization, verification, ... modes)

```
{
  "version": "1.2",
  "variables": [
    {"label":"y1", "interface":"output", "type":"real"},
    {"label":"y2", "interface":"output", "type":"real"},
    {"label":"x1", "interface":"input", "type":"real", "range":[0,10]},
    {"label":"x2", "interface":"input", "type":"int", "range":[-1,1]},
    {"label":"p1", "interface":"knob", "type":"real", "range":[0,10], "rad-rel":0.1, "grid":[2,4,7]},
    {"label":"p2", "interface":"knob", "type":"int", "range":[3,7], "rad-abs":0.2}
  ],
  "alpha": "p2<5 and x1==10 and x2<12",
  "beta": "y1>=4 and y2==8",
  "eta": "p1==4 or (p1==8 and p2 > 3)",
  "assertions": {
    "assert1": "(y2**3+p2)/2>6",
    "assert2": "y1>=0",
    "assert3": "y2>0"
  },
  "objectives": {
    "objective1": "(y1+y2)/2",
    "objective2": "y1"
  }
}
```



Supported Platforms

- Native build
  - Ubuntu 22.04
  - Ubuntu 24.04
- Docker containers
  - x64 - Intel and AMD CPUs
  - aarch64 - Mac M series CPUs